RabbitMQ + MassTransit - Complete Guide Using Your Project

Part 1: The Problem RabbitMQ Solves

Imagine you have 6 friends working together on a project:

Without RabbitMQ (Direct HTTP Calls):

```
Basket Service → Calls Ordering Service directly
                ↓
If Ordering Service is down → Basket Service fails
If Ordering Service is slow → Basket Service waits (blocks user)
If 1000 users checkout → Ordering Service gets overwhelmed
```

Problems:

- **Tight Coupling**: Services depend directly on each other
- **Blocking**: One slow service blocks everything
- **No Buffering**: If service is busy, requests are lost
- **No Reliability**: If service is down, messages are lost

With RabbitMQ (Message Broker):

```
Basket Service → Sends message to RabbitMQ
                ↓
RabbitMQ stores the message (safe buffer)
                ↓
Ordering Service picks up message when ready
                ↓
If Ordering Service is down → RabbitMQ keeps message
If Ordering Service is slow → RabbitMQ queues messages
If 1000 messages → RabbitMQ handles the load
```

Benefits:

Decoupled: Services don't know about each other
Async: Services don't wait for each other
Buffering: RabbitMQ handles traffic spikes
Reliable: Messages aren't lost if service is down
Scalable: Multiple consumers can process messages

Part 2: RabbitMQ Configuration in Docker Compose

Your docker-compose.yml (Lines 17-18)

```
rabbitmq:
  image: rabbitmq:3-management-alpine
```

What this does:

Defines RabbitMQ service
Uses `rabbitmq:3-management-alpine` image (includes management UI)
Alpine = lightweight Linux distribution

Your docker-compose.override.yml (Lines 65-70)

```
rabbitmq:
  container_name: rabbitmq
  restart: always
  ports:
    - "5672:5672"      # AMQP protocol (message communication)
    - "15672:15672"    # Management UI (web interface)
```

Port Explanation:

5672: Main port for AMQP protocol (services connect here to send/receive messages)
15672: Management UI port (web interface to monitor RabbitMQ)

Access Management UI:

Open browser: http://localhost:15672
Default credentials: `guest` / `guest`

Environment Variables for Services

Basket Service (Lines 87-96):

```
basket:
  environment:
    -
"EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - basketdb
    - rabbitmq
```

Ordering Service (Lines 111-121):

```
ordering:
  environment:
    -
"EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - orderdb
    - rabbitmq
```

Payment Service (Lines 123-131):

```
payment:
  environment:
    -
"EventBusSettings__HostAddress=amqp://guest:guest@rabbitmq:5672/"
  depends_on:
    - rabbitmq
```

Connection String Breakdown:

```
amqp://guest:guest@rabbitmq:5672/
  ↓      ↓      ↓         ↓
protocol user  password host   port
```

Note: `rabbitmq` is the Docker container name (Docker internal DNS)

Part 3: MassTransit Implementation in Services

MassTransit is a .NET library that makes it easy to work with message brokers like RabbitMQ.

Step 1: Install NuGet Packages

Basket.Application/Basket.Application.csproj (Line 11):

```
<<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />
```

Ordering.Application/Ordering.Application.csproj (Line 16):

```
<<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />
```

Payment/Payment.csproj (Lines 15-16):

```
<<PackageReference Include="MassTransit" Version="8.5.2" />
<<PackageReference Include="MassTransit.RabbitMQ" Version="8.5.2" />
```

Step 2: Configure MassTransit in Program.cs

Basket Service (Publisher Only)

File: Basket.API/Program.cs (Lines 72-80)

```
//Add mass Transit
builder.Services.AddMassTransit(config =>
{
    config.UsingRabbitMq((ct, cfg) =>
    {

cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
    });
```

```
});
```

What this does:

Registers MassTransit services
Configures RabbitMQ as the message broker
Reads connection string from configuration

Ordering Service (Publisher + Consumer)

File: Ordering.API/Program.cs (Lines 26-54)

```
//Mass Transit
builder.Services.AddMassTransit(config =>
{
    //Mark as Consumer
    config.AddConsumer<BasketOrderingConsumer>();
    config.AddConsumer<<PaymentCompletedConsumer>();
    config.AddConsumer<<PaymentFailedConsumer>();

    config.UsingRabbitMq((ctx, cfg) =>
    {

cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
        cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));

        //provide the queue name with consumer settings
        cfg.ReceiveEndpoint(EventBusConstant.BasketCheckoutQueue, c =>
        {
            cfg.UseRetry(r => r.Interval(5,
TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<BasketOrderingConsumer>(ctx);
        });

        cfg.ReceiveEndpoint(EventBusConstant.PaymentCompleteQueue, c
=>
        {
            cfg.UseRetry(r => r.Interval(5,
TimeSpan.FromSeconds(10)));
```

```
            c.ConfigureConsumer<<PaymentCompletedConsumer>(ctx);
        });

        cfg.ReceiveEndpoint(EventBusConstant.PaymentFailedQueue, c =>
        {
            cfg.UseRetry(r => r.Interval(5,
TimeSpan.FromSeconds(10)));
            c.ConfigureConsumer<<PaymentFailedConsumer>(ctx);
        });
    });
});
```

Breakdown:

Register Consumers: `AddConsumer<T>()` - Tells MassTransit which classes consume messages
Configure RabbitMQ: `UsingRabbitMq()` - Sets up RabbitMQ connection
Add Retry: `UseRetry()` - Retries failed messages (5 times, 10-second intervals)
Create Queues: `ReceiveEndpoint()` - Creates queues for each consumer
Bind Consumers: `ConfigureConsumer<T>()` - Links consumer to queue

Payment Service (Consumer + Publisher)

File: Payment/Program.cs (Lines 13-27)

```
// MassTransit
builder.Services.AddMassTransit(config =>
{
    config.AddConsumer<OrderCreatedConsumer>();
    config.UsingRabbitMq((ctx, cfg) =>
    {

cfg.Host(builder.Configuration["EventBusSettings:HostAddress"]);
        cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)));

        cfg.ReceiveEndpoint(EventBusConstant.OrderCreatedQueue, c =>
        {
            c.UseMessageRetry(r => r.Interval(5,
TimeSpan.FromSeconds(10)));
```

```
            c.ConfigureConsumer<OrderCreatedConsumer>(ctx);
        });
    });
});
```

Part 4: Event Definitions (Shared Library)

EventBusConstant (Queue Names)

File: Infrastructure/EventBus.Messages/Common/EventBusConstant.cs

```
public class EventBusConstant
{
    public const string BasketCheckoutQueue = "basketcheckout-queue";
    public const string OrderCreatedQueue = "ordercreated-queue";
    public const string PaymentCompleteQueue = "paymentcompleted-
queue";
    public const string PaymentFailedQueue = "paymentfailed-queue";
}
```

Why constants?:

- Centralized queue name management
- Avoids typos
- Easy to rename queues in one place

Event Classes

BasketCheckoutEvent (Already read from previous session):

```
public record BasketCheckoutEvent : BaseIntegrationEvent
{
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    // ... other properties
}
```

OrderCreatedEvent (Already read from previous session):

```
public record OrderCreatedEvent : BaseIntegrationEvent
{
    public int Id { get; set; }
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    // ... other properties
}
```

PaymentCompletedEvent (Already read from previous session):

```
public record PaymentCompletedEvent : BaseIntegrationEvent
{
    public int OrderId { get; set; }
    public string UserName { get; set; }
    public decimal TotalPrice { get; set; }
    public Guid CorrelationId { get; set; }
}
```

PaymentFailedEvent (Already read from previous session):

```
public record PaymentFailedEvent : BaseIntegrationEvent
{
    public int OrderId { get; set; }
    public string UserName { get; set; }
    public string Reason { get; set; }
    public Guid CorrelationId { get; set; }
}
```

Part 5: Event Flow - Complete Example

The Complete Checkout Flow

```
User clicks "Checkout"
    ↓
Basket Service receives request
```

```
    ↓
Basket Service publishes BasketCheckoutEvent to RabbitMQ
    ↓
Ordering Service consumes BasketCheckoutEvent
    ↓
Ordering Service creates order in database
    ↓
Ordering Service publishes OrderCreatedEvent to RabbitMQ
    ↓
Payment Service consumes OrderCreatedEvent
    ↓
Payment Service processes payment
    ↓
Payment Service publishes PaymentCompletedEvent OR PaymentFailedEvent
    ↓
Ordering Service consumes PaymentCompletedEvent/FailedEvent
    ↓
Ordering Service updates order status
    ↓
Basket Service deletes shopping basket
```

Step-by-Step Code Flow

Step 1: User Checkout (Basket Service)

File: Basket.Application/Handlers/BasketCheckoutHandler.cs (Lines 24-40)

```
public async Task<Unit> Handle(BasketCheckoutCommand request,
CancellationToken cancellationToken)
{
    var basketDto = request.BasketCheckoutDto;

    // 1. Get the basket using MediatR
    var basketResponse = await _mediator.Send(new
GetBasketByUserNameQuery(basketDto.UserName), cancellationToken);

    if (basketResponse is null || !basketResponse.Items.Any())
    {
```

```
        throw new InvalidOperationException("Basket not found or
empty");
    }

    // 2. Convert to entity
    var basket = basketResponse.ToEntity();

    // 3. Map to event
    var evt = basketDto.ToBasketCheckoutEvent(basket);

    // 4. Publish event to RabbitMQ
    _logger.LogInformation("Publishing BasketCheckoutEvent For
{User}", basket.UserName);
    await _publishEndpoint.Publish(evt, cancellationToken);

    // 5. Delete the basket
    await _mediator.Send(new
DeleteBasketByUserNameCommand(basket.UserName), cancellationToken);

    return Unit.Value;
}
```

Key line: `await _publishEndpoint.Publish(evt, cancellationToken);`

Sends BasketCheckoutEvent to RabbitMQ
MassTransit handles the connection and serialization

Step 2: Ordering Service Consumes Basket Checkout

File: Ordering.Application/EventBusConsumer/BasketOrderingConsumer.cs

```
public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>
{
    private readonly ICommandHandler<<CreateOrderCommand, int>
_createOrderHandler;
    private readonly ILogger<BasketOrderingConsumer> _logger;

    public BasketOrderingConsumer(ICommandHandler<<CreateOrderCommand,
int> createOrderHandler, ILogger<BasketOrderingConsumer> logger)
```

```
    {
        _createOrderHandler = createOrderHandler;
        _logger = logger;
    }

    public async Task Consume(ConsumeContext<BasketCheckoutEvent>
context)
    {
        using var scopr = _logger.BeginScope("Consuming Basket
Checkout Event for {CorrelationId}", context.Message.CorrelationId);

        // Convert event to command
        var command = context.Message.ToCheckoutOrderCommand();

        // Create order in database
        var orderId = await _createOrderHandler.Handle(command,
context.CancellationToken);

        _logger.LogInformation("Basket checkout Event Completed
Successfully !! OrderId: {OrderId}", orderId);
    }
}
```

What happens:

MassTransit automatically calls this when message arrives in queue
`ConsumeContext<BasketCheckoutEvent>` contains the message
Converts event to command
Calls command handler to create order
Order is saved to database

Step 3: Ordering Service Publishes Order Created

After creating order, Ordering service publishes OrderCreatedEvent (this would be in the order creation handler, not shown in the files I read, but follows the same pattern as Basket checkout).

Step 4: Payment Service Consumes Order Created

File: Payment/EventBusConsumer/OrderCreatedConsumer.cs

```
public class OrderCreatedConsumer : IConsumer<OrderCreatedEvent>
{
    private readonly IPublishEndpoint _publishEndpoint;
    private readonly ILogger<OrderCreatedConsumer> _logger;

    public OrderCreatedConsumer(IPublishEndpoint publishEndpoint,
ILogger<OrderCreatedConsumer> logger)
    {
        _publishEndpoint = publishEndpoint;
        _logger = logger;
    }

    public async Task Consume(ConsumeContext<OrderCreatedEvent>
context)
    {
        var message = context.Message;
        _logger.LogInformation("Processing Payment for Order Id :
{@OrderId} ", message.Id);

        // Simulate Payment Processing
        await Task.Delay(1000);

        if (message.TotalPrice > 0)
        {
            // Payment Success
            var completedEvent = new PaymentCompletedEvent
            {
                OrderId = message.Id,
                CorrelationId = context.CorrelationId.Value,
            };
            await _publishEndpoint.Publish(completedEvent);
            _logger.LogInformation("Payment success for Order Id :
{@Order Id} and Correlation Id : {@CorrelationId}", message.Id,
message.CorrelationId);
        }
```

```
        else
        {
            // Payment Failed
            var failedEvent = new PaymentFailedEvent
            {
                OrderId = message.Id,
                CorrelationId = context.CorrelationId.Value,
                Reason = "Total Price was zero or negative"
            };
            await _publishEndpoint.Publish(failedEvent);
            _logger.LogWarning("Payment Failed for Order Id : {@Order
Id} and Correlation Id : {@CorrelationId}", message.Id,
message.CorrelationId);
        }
    }
}
```

What happens:

Consumes OrderCreatedEvent
Processes payment (simulated with `Task.Delay`)
If success → publishes PaymentCompletedEvent
If failure → publishes PaymentFailedEvent

Step 5: Ordering Service Consumes Payment Result

File: Ordering.Application/EventBusConsumer/PaymentCompletedConsumer.cs (not read, but would follow same pattern)

This consumer would update the order status in the database based on payment result.

Part 6: Queue Architecture

```
RabbitMQ Server
├── basketcheckout-queue
│   └── Ordering Service (BasketOrderingConsumer)
├── ordercreated-queue
│   └── Payment Service (OrderCreatedConsumer)
```

```
├── paymentcompleted-queue
│   └── Ordering Service (PaymentCompletedConsumer)
└── paymentfailed-queue
    └── Ordering Service (PaymentFailedConsumer)
```

Part 7: Key Concepts

Publisher vs Consumer

Publisher (Basket Service):

- Sends messages to RabbitMQ
- Uses `IPublishEndpoint`
- Example: `await _publishEndpoint.Publish(event)`

Consumer (Ordering, Payment Services):

- Receives messages from RabbitMQ
- Implements `IConsumer<T>`
- Example: `public class BasketOrderingConsumer : IConsumer<BasketCheckoutEvent>`

Correlation ID

Purpose: Track a request across multiple services

Example:

```
Basket Checkout (CorrelationId: abc-123)
    ↓ publish
RabbitMQ
    ↓ consume
Ordering Service (CorrelationId: abc-123)
    ↓ publish
RabbitMQ
    ↓ consume
Payment Service (CorrelationId: abc-123)
```

All events for one checkout have the same CorrelationId, making it easy to trace the entire flow.

Retry Policy

Your configuration: `cfg.UseRetry(r => r.Interval(5, TimeSpan.FromSeconds(10)))`

What it does:

If message processing fails, retry up to 5 times
Wait 10 seconds between retries
After 5 failures, move message to dead-letter queue

Why this is important:

Temporary failures (database timeout, network glitch) don't cause message loss
Gives services time to recover
Dead-letter queue for manual inspection of failed messages

Summary

RabbitMQ in Your Project: Your ECommerce microservices use RabbitMQ as a message broker to enable asynchronous, event-driven communication between services.

Your Implementation:

RabbitMQ Server: Configured in docker-compose.yml (lines 17-18) with management UI
Ports: 5672 (AMQP), 15672 (Management UI)
Connection String: `amqp://guest:guest@rabbitmq:5672/`
Libraries: MassTransit 8.5.2 + MassTransit.RabbitMQ in Basket, Ordering, Payment services

Services Using RabbitMQ:

Basket: Publisher (sends BasketCheckoutEvent)
Ordering: Consumer (BasketCheckoutEvent) + Publisher (OrderCreatedEvent) + Consumer (PaymentCompletedEvent, PaymentFailedEvent)

Payment: Consumer (OrderCreatedEvent) + Publisher (PaymentCompletedEvent, PaymentFailedEvent)

Event Flow:

```
Basket Checkout → RabbitMQ → Ordering (creates order) → RabbitMQ → Payment (processes payment) → RabbitMQ → Ordering (updates status)
```

Key Features Implemented:

- Queue definitions in EventBusConstant.cs
- Retry policy (5 retries, 10-second intervals)
- Correlation ID for request tracking
- Separate queues for each event type

Your implementation follows best practices for event-driven microservices architecture with proper decoupling, reliability, and scalability.